Module-8

Topic-1:Transfer learning:What, Why, and When?

Acknowledgement

Thanks to the following courses and corresponding researchers for making their teaching/research material online

- Mike X Cohen's Master Deep Learning course (most contents taken from this Udemy Course)
- AI Tools such as Chat GPT, Grok, etc
- Introduction to Deep Learning, University of Illinois at Urbana-Champaign
- Introduction to Deep Learning, Carnegie Mellon University
- Convolutional Neural Networks for Visual Recognition, Stanford University
- Natural Language Processing with Deep Learning, Stanford University
- And Many More .....

Topic -1

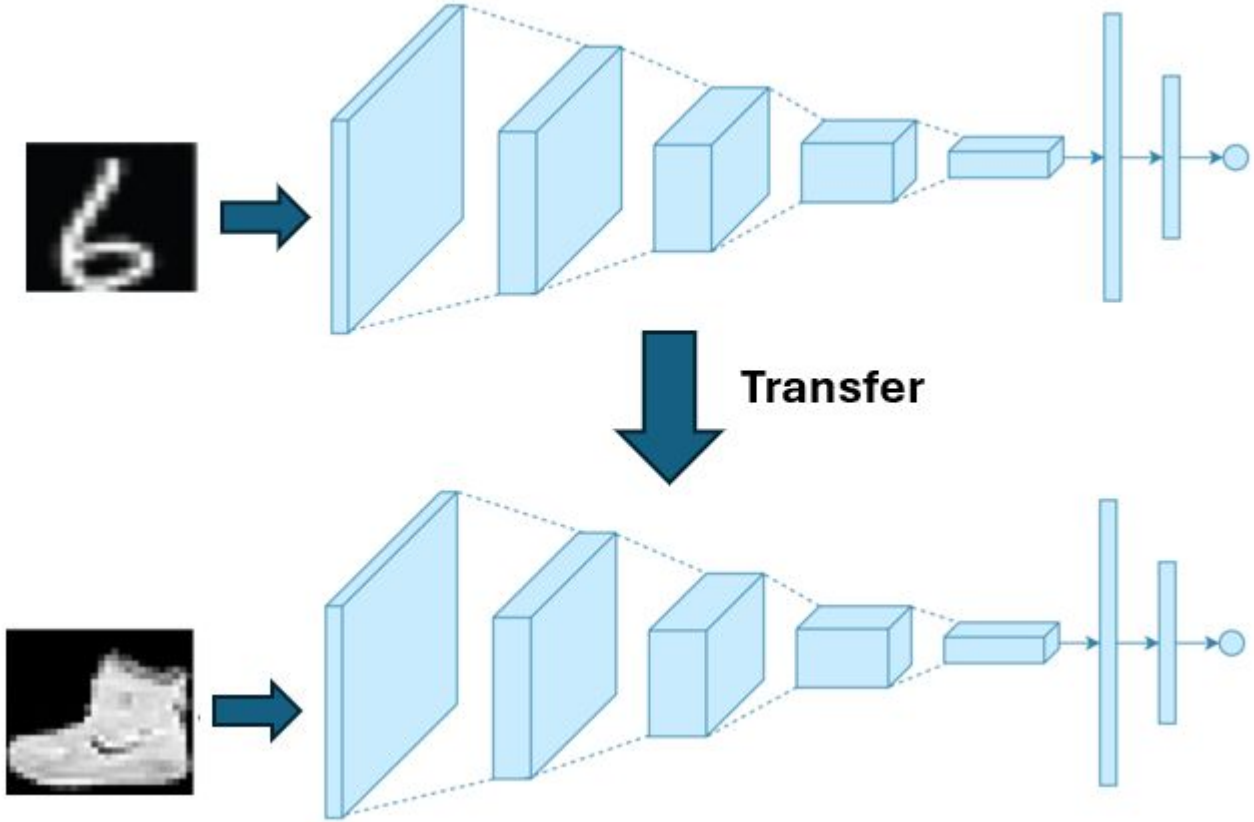
Transfer learning:What, Why, and When?

Transfer Learning; What, Why, and when?

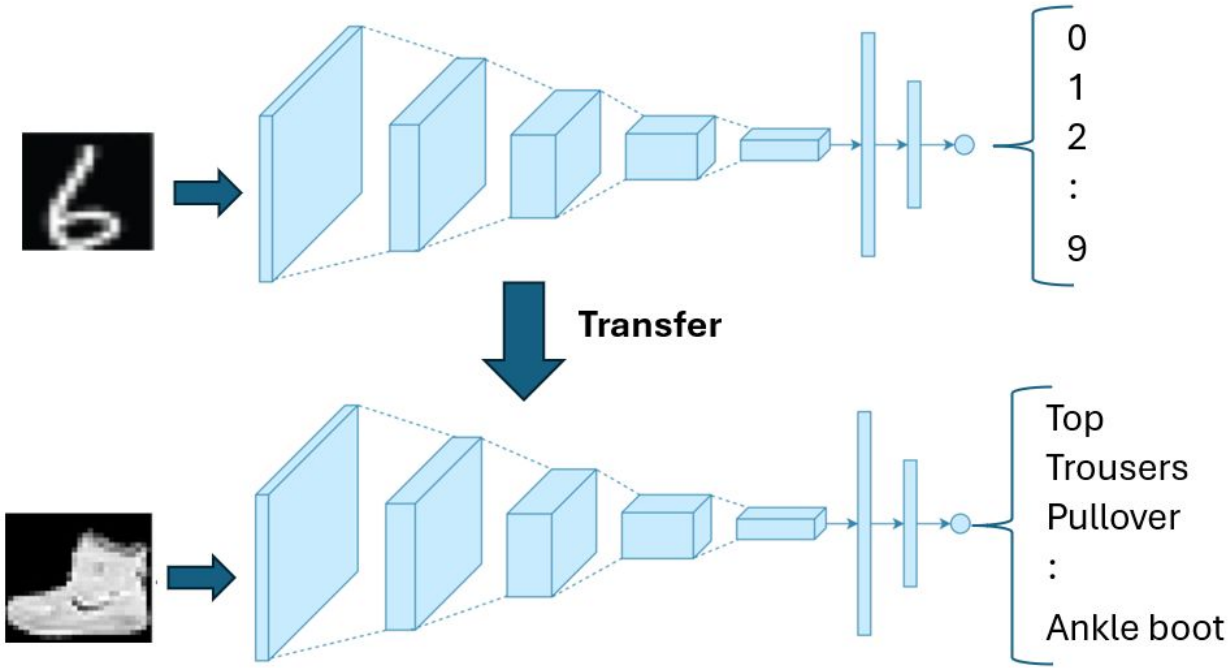
In this topic, you will learn

- What “transfer learning” means.
- Why transfer learning is useful in DL.
- When to use transfer learning
- How to use transfer learning

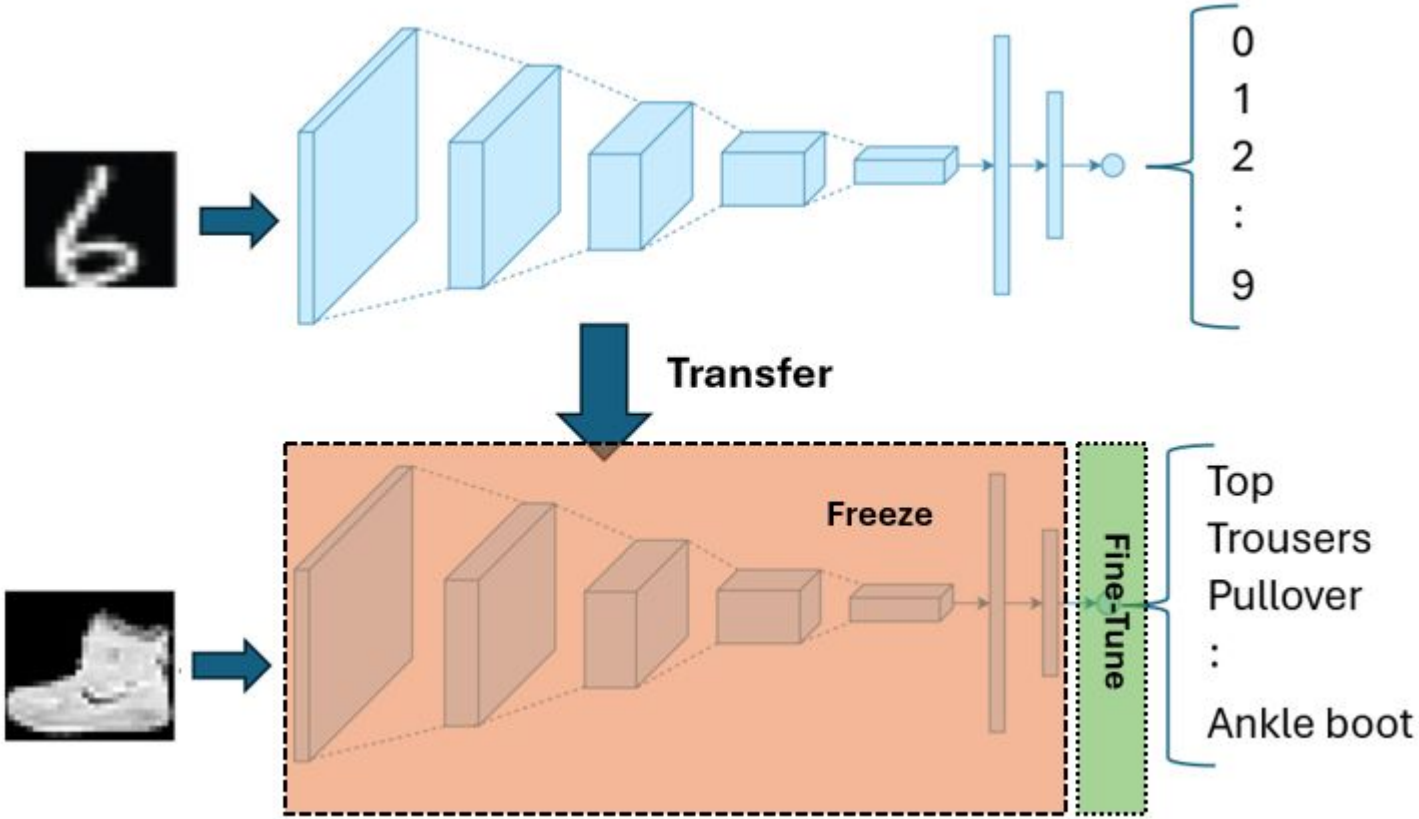
What is transfer learning?



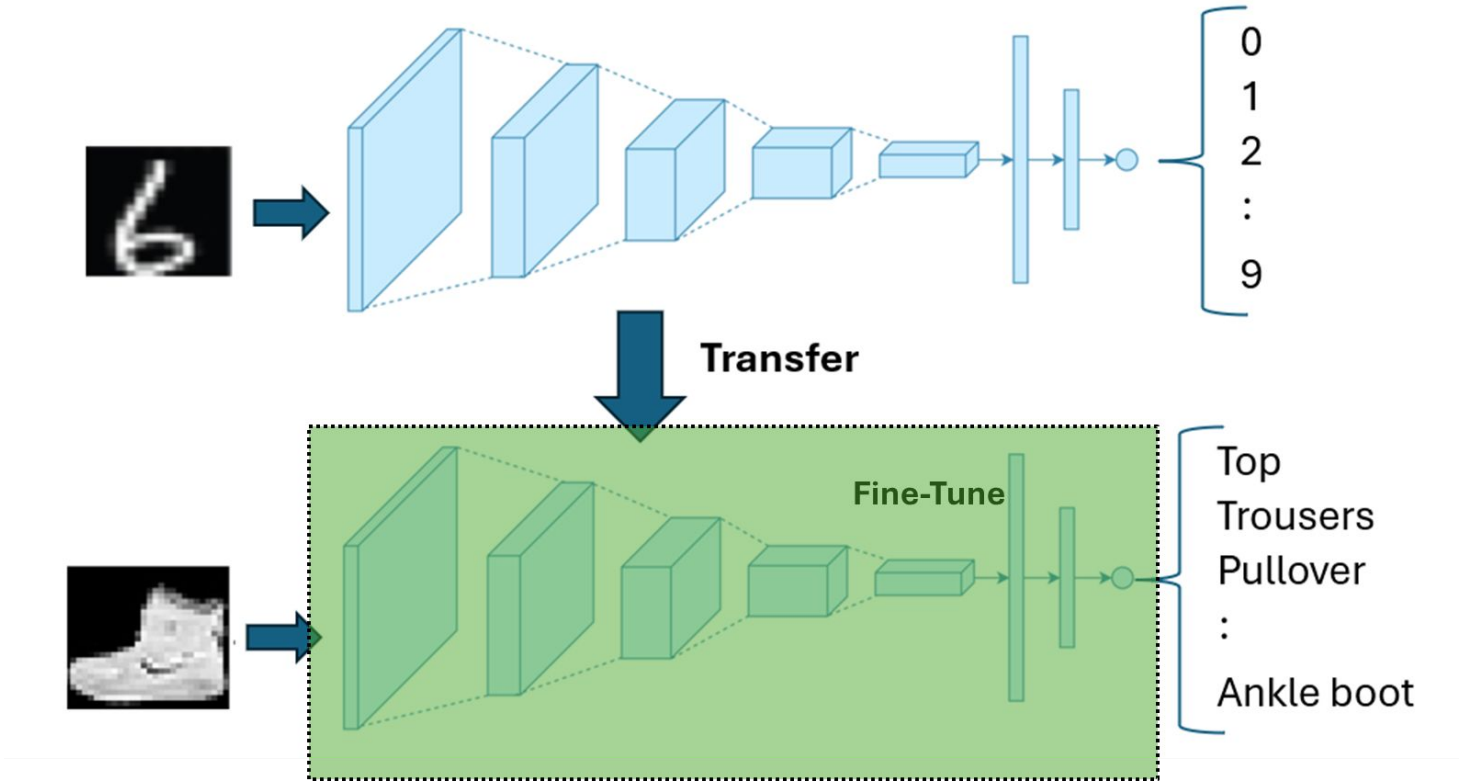
What is transfer learning?



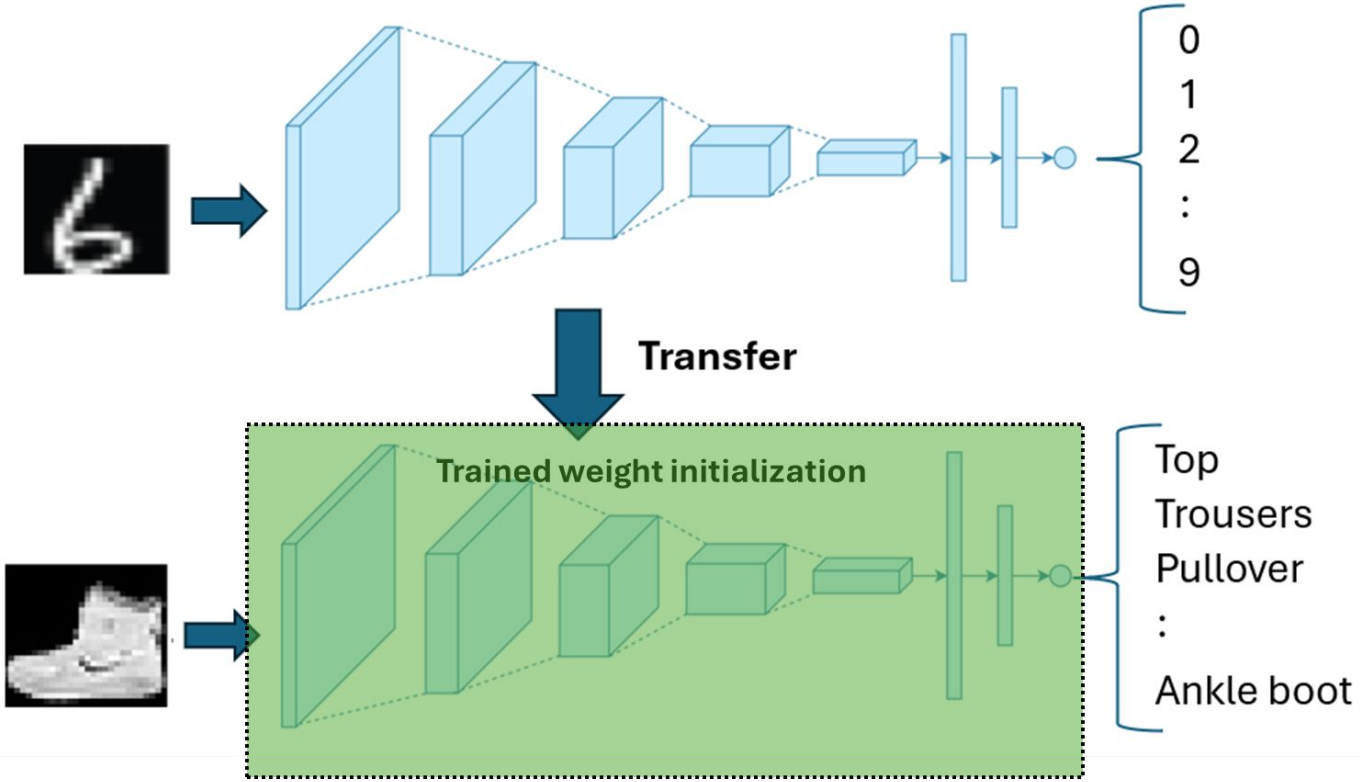
What is transfer learning?



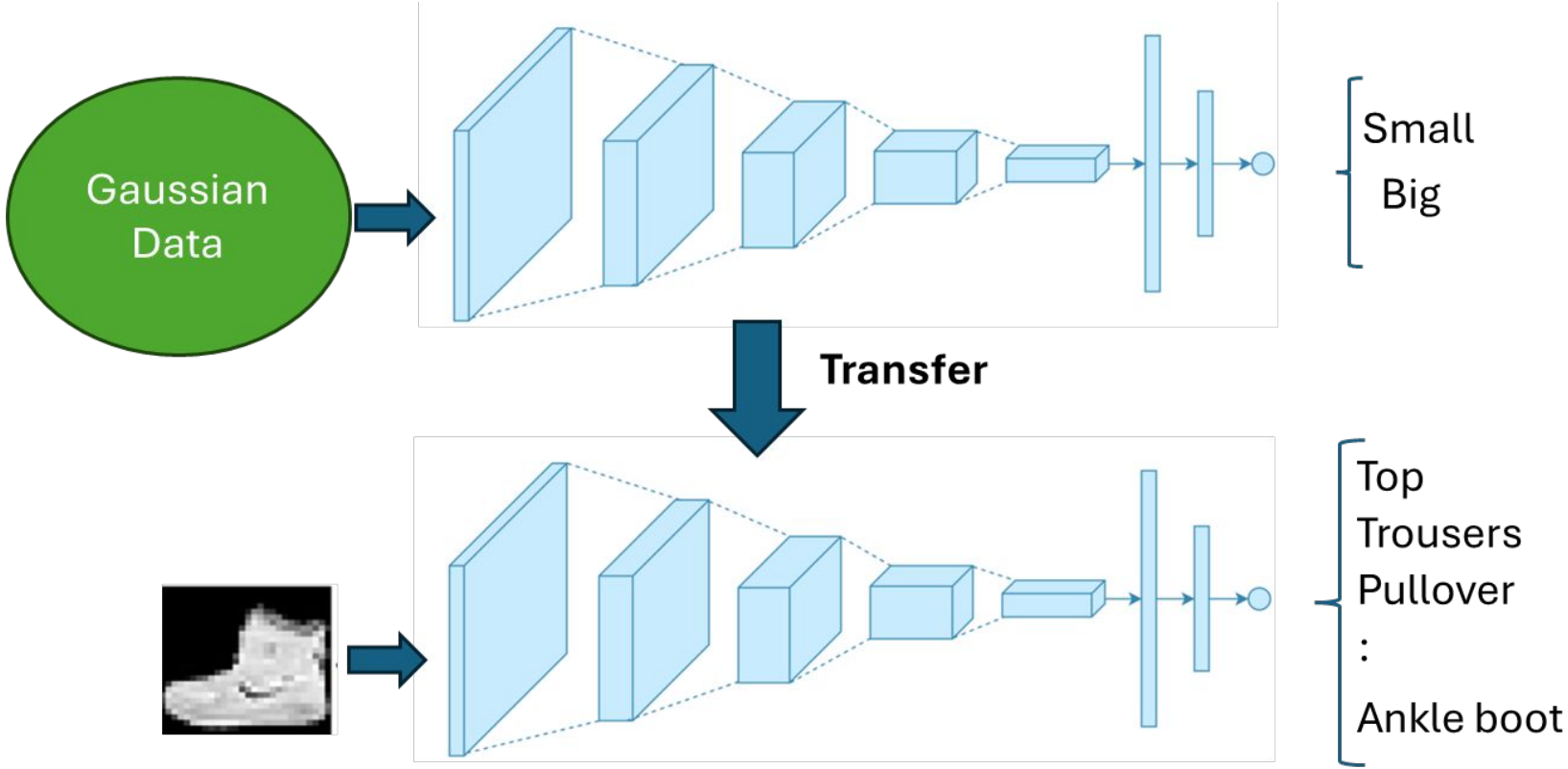
What is transfer learning?



What is transfer learning?



What is transfer learning?



Why is transfer learning useful?

Transfer learning allows you to

- Leverage HUGE datasets and existing (source) models to solve new problems.
- Fine-tune existing trained models to your unique data.
- Deploy models quickly and with limited computational and data resources.

When is transfer learning appropriate?

Use transfer learning when

- Your problem is similar to a problem that someone else has solved (and published their model)
- The model was trained on a lot more data than what you have.
- The model is deep;shallow models are less likely to transfer well unless the target data are very similar.

Is transfer learning appropriate here?

- Fine-tune AlexNet to distinguish different models of BMWs(right)
- Fine-tune AlexNet to predict credit card fraud(wrong)
- Train a model on US housing sales data and then fine-tune that model to India housing sales data(don't' know).

How to use transfer learning

- Decide whether it is appropriate:(1) Do you have insufficient complex data and (2) does a well-trained model exist?
- Decide which(if any) layers to freeze: The closer your data to the source model, the more layers you can freeze.
- Fine-tune the target model as much as possible.
- Inspect the model and results carefully; never assume that transfer will be successful.

Transfer Learning; What, Why, and when?

In this topic, you have learned about

- What “transfer learning” means.
- Why transfer learning is useful in DL.
- When to use transfer learning
- How to use transfer learning

Topic -2

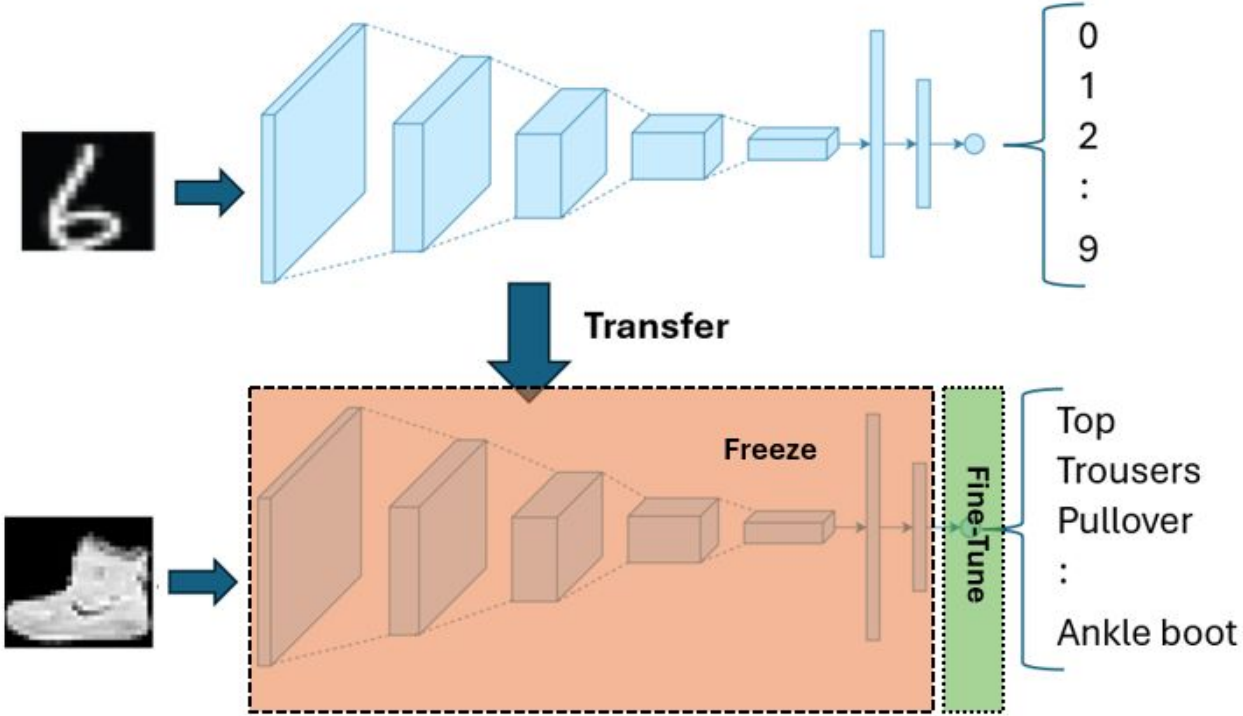
Transfer learning:MNIST->FMNIST

Transfer Learning; What, Why, and when?

In this topic, you will learn

- How to implement transfer learning in PyTorch.
- How to copy model parameters

What is transfer learning?



A-Code

Transfer Learning; What, Why, and when?

In this topic, you have learned about

- How to implement transfer learning in PyTorch.
- How to copy model parameters

Topic -3

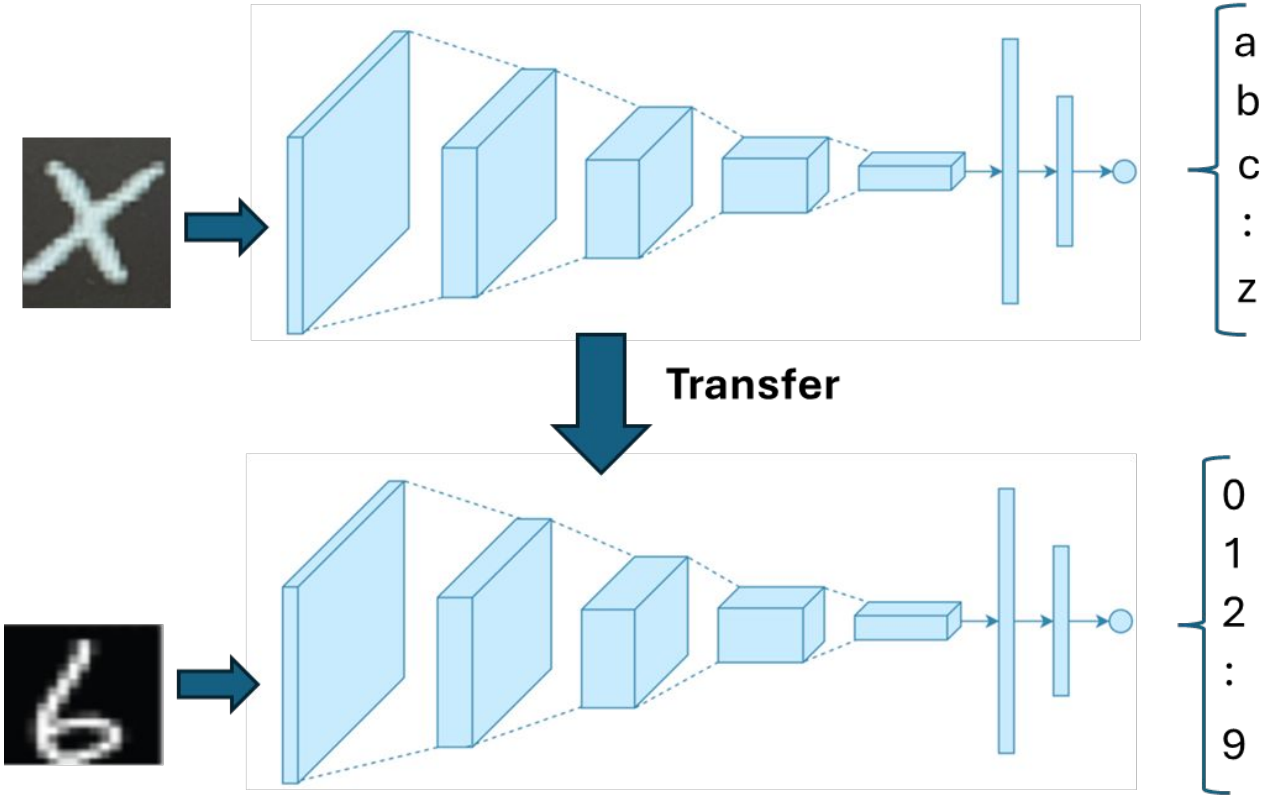
Transfer learning:Letters to numbers

Transfer Learning; What, Why, and when?

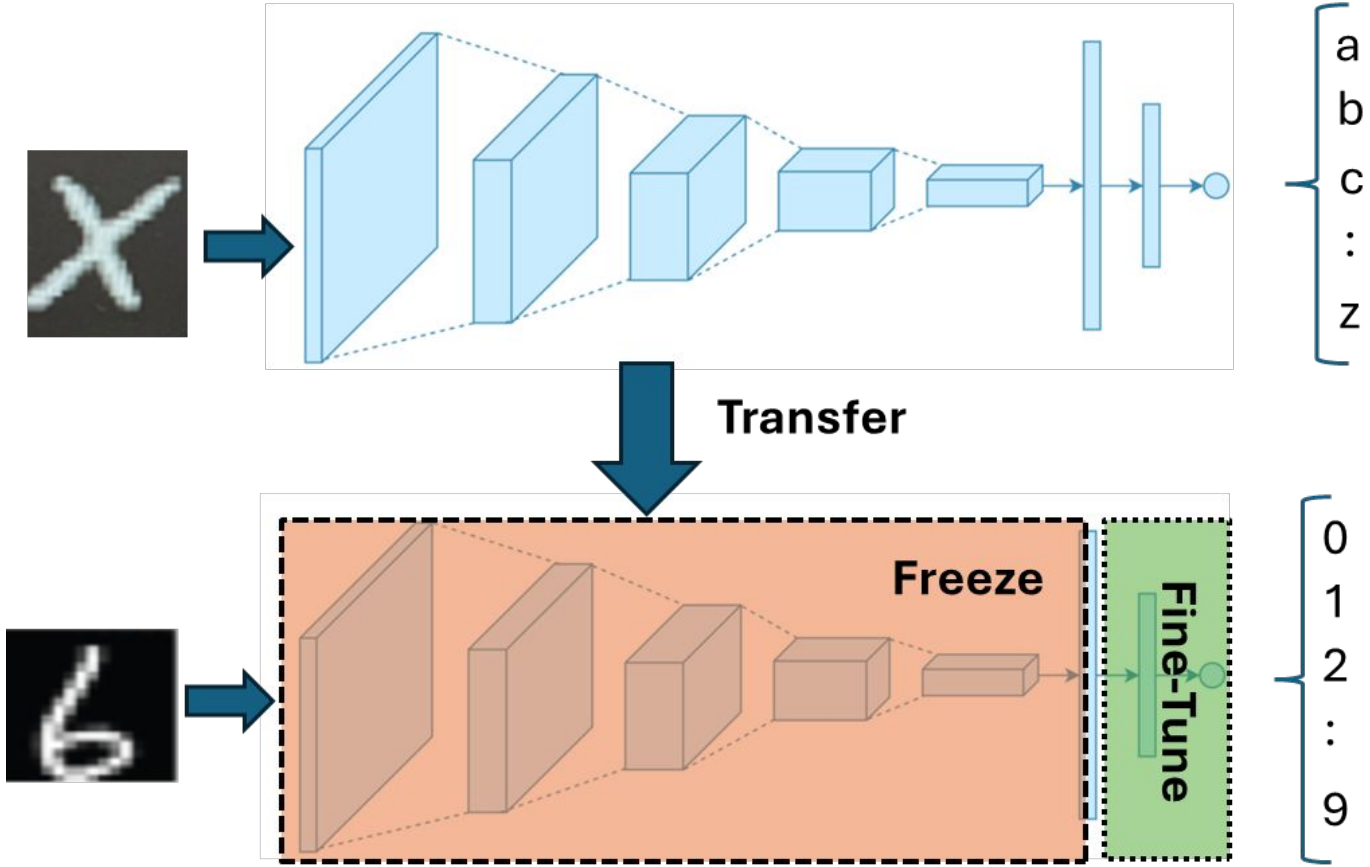
In this topic, you will learn

- Gain more experience working with transfer learning
- Learn how to freeze only the categorization layers in re-training

What is transfer learning ?



What is transfer learning ?



Our Objective

- Import both ‘letter’ and ‘digits’
- Train on letters for 5 epochs
- Transfer the model to digits
- Fine-tune (1) the entire model, and (2) only the categorization layers. Train only one epoch.

A-code

Transfer Learning; What, Why, and when?

In this topic, you have explored how to

- Gain more experience working with transfer learning
- Learn how to freeze only the categorization layers in re-training

Topic -4

Transfer learning:famous CNN architectures

Transfer learning:famous CNN architectures

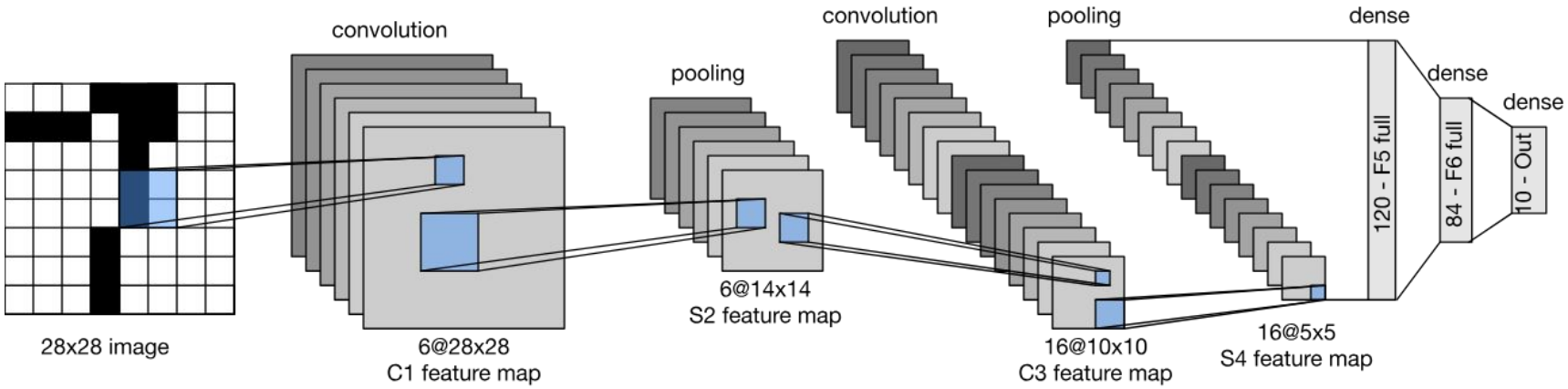
In this topic, you will explore

- Different architecture of CNN models

TABLE 1. State of art CNN models.

Model	Proposed Year	No of layers	No of Parameters
AlexNet	2012	8	62.3 M
DenseNet121	2016	121	6.99 M
EfficientNet_b0	2019	66	5.3 M
GoogLeNet	2014	22	5.64 M
MnasNet1_0	2018	39	4.4 M
MobileNet_v1	2017	28	3.25 M
Resnet 50	2015	50	23.59 M
ShuffleNet_v1	2018	27	5.4 M
VGG-16	2014	16	134.42 M
VGG-19	2014	19	143.67 M

LeNet


convolution
pooling
convolution
pooling
dense
dense
dense
28x28 image
6@28x28
C1 feature map
6@14x14
S2 feature map
16@10x10
C3 feature map
16@5x5
S4 feature map
120 - F5 full
84 - F6 full
10 - Out

AlexNet

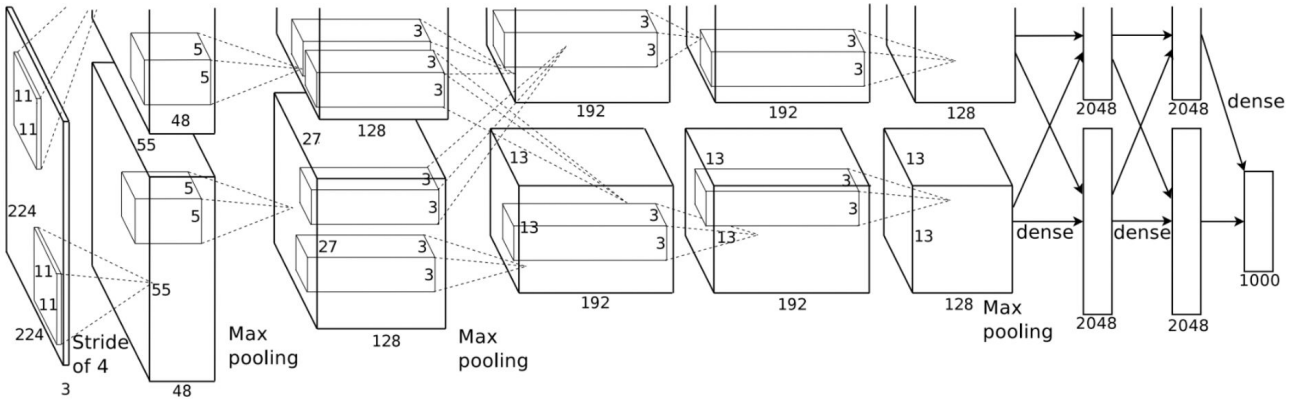
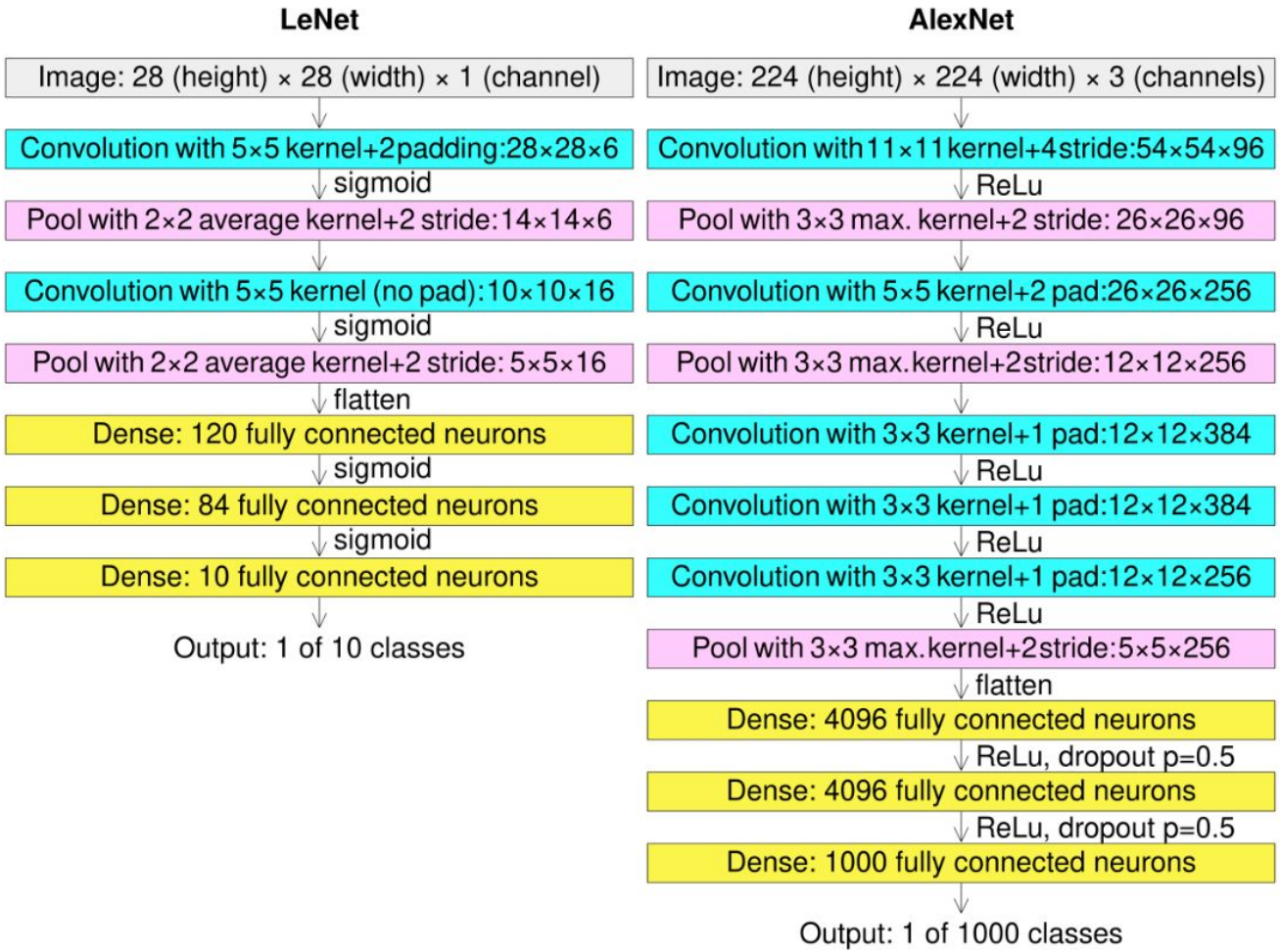
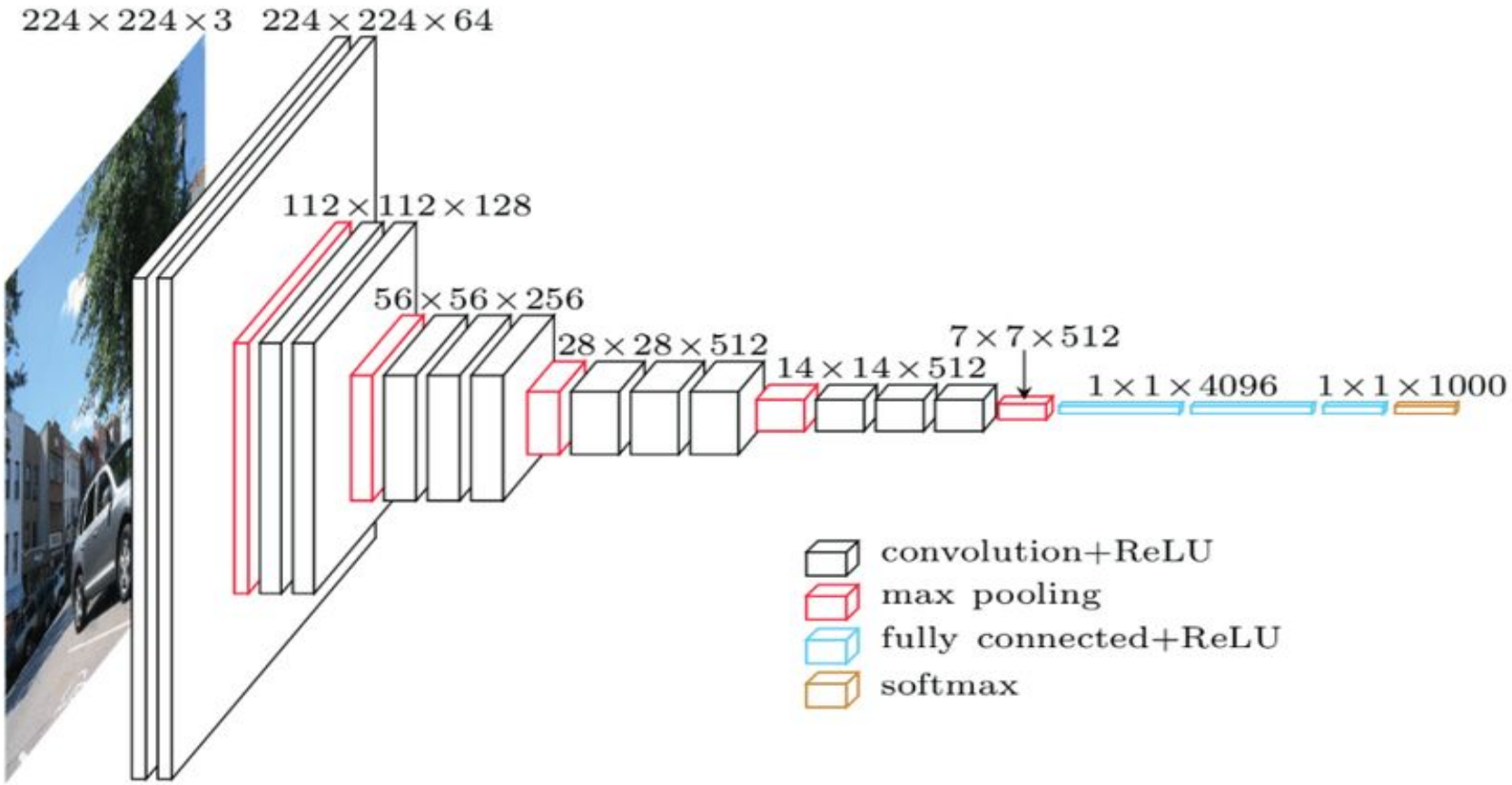
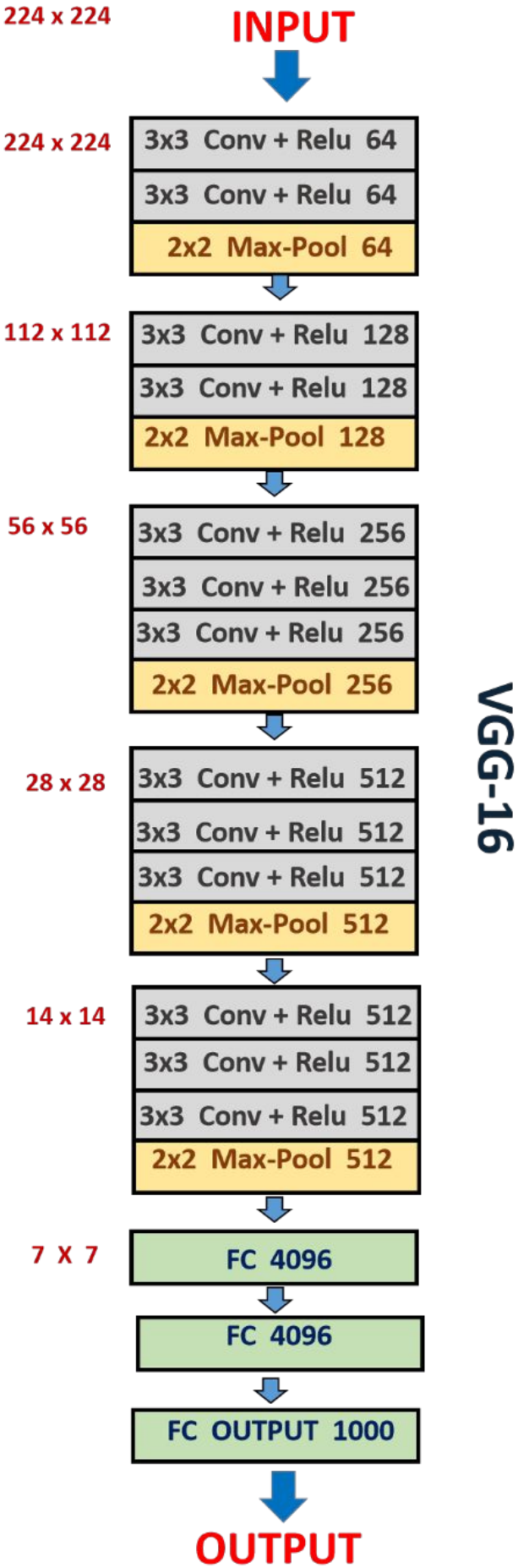


Figure 2: An illustration of the architecture of our CNN, explicitly showing the delineation of responsibilities between the two GPUs. One GPU runs the layer-parts at the top of the figure while the other runs the layer-parts at the bottom. The GPUs communicate only at certain layers. The network's input is 150,528-dimensional, and the number of neurons in the network's remaining layers is given by 253,440–186,624–64,896–64,896–43,264–4096–4096–1000.

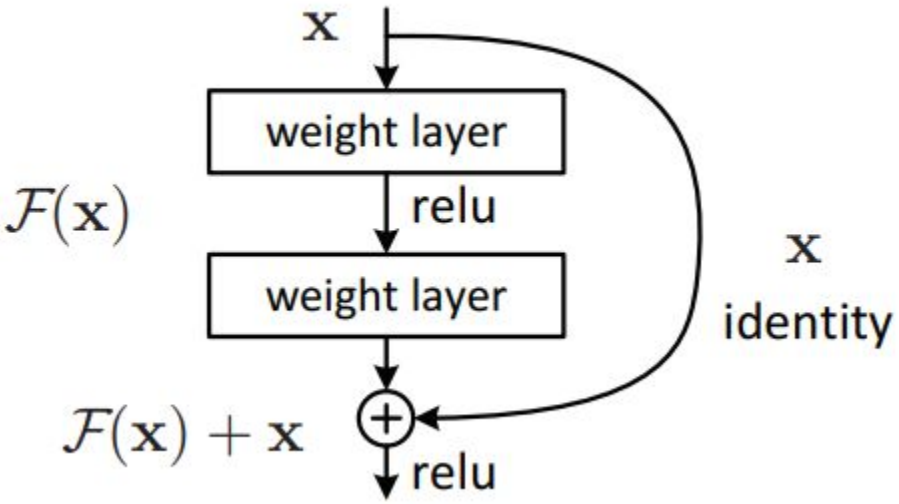

LeNet
Image: 28 (height) × 28 (width) × 1 (channel)
Convolution with 5×5 kernel+2padding:28×28×6
sigmoid
Pool with 2×2 average kernel+2 stride:14×14×6
Convolution with 5×5 kernel (no pad):10×10×16
sigmoid
Pool with 2×2 average kernel+2 stride: 5×5×16
flatten
Dense: 120 fully connected neurons
sigmoid
Dense: 84 fully connected neurons
sigmoid
Dense: 10 fully connected neurons
Output: 1 of 10 classes
AlexNet
Image: 224 (height) × 224 (width) × 3 (channels)
Convolution with 11×11 kernel+4 stride:54×54×96
ReLu
Pool with 3×3 max. kernel+2 stride: 26×26×96
Convolution with 5×5 kernel+2 pad:26×26×256
ReLu
Pool with 3×3 max.kernel+2stride:12×12×256
Convolution with 3×3 kernel+1 pad:12×12×384
ReLu
Convolution with 3×3 kernel+1 pad:12×12×384
ReLu
Convolution with 3×3 kernel+1 pad:12×12×256
ReLu
Pool with 3×3 max.kernel+2stride:5×5×256
flatten
Dense: 4096 fully connected neurons
ReLu, dropout p=0.5
Dense: 4096 fully connected neurons
ReLu, dropout p=0.5
Dense: 1000 fully connected neurons
Output: 1 of 1000 classes

VGGNet


224 × 224 × 3
224 × 224 × 64
112 × 112 × 128
56 × 56 × 256
28 × 28 × 512
14 × 14 × 512
7 × 7 × 512
1 × 1 × 4096
1 × 1 × 1000
convolution+ReLU
max pooling
fully connected+ReLU
softmax


224 x 224
INPUT
224 x 224
3x3 Conv + Relu 64
3x3 Conv + Relu 64
2x2 Max-Pool 64
112 x 112
3x3 Conv + Relu 128
3x3 Conv + Relu 128
2x2 Max-Pool 128
56 x 56
3x3 Conv + Relu 256
3x3 Conv + Relu 256
3x3 Conv + Relu 256
2x2 Max-Pool 256
28 x 28
3x3 Conv + Relu 512
3x3 Conv + Relu 512
3x3 Conv + Relu 512
2x2 Max-Pool 512
14 x 14
3x3 Conv + Relu 512
3x3 Conv + Relu 512
3x3 Conv + Relu 512
2x2 Max-Pool 512
7 X 7
FC 4096
FC 4096
FC OUTPUT 1000
OUTPUT
VGG-16

ResNet



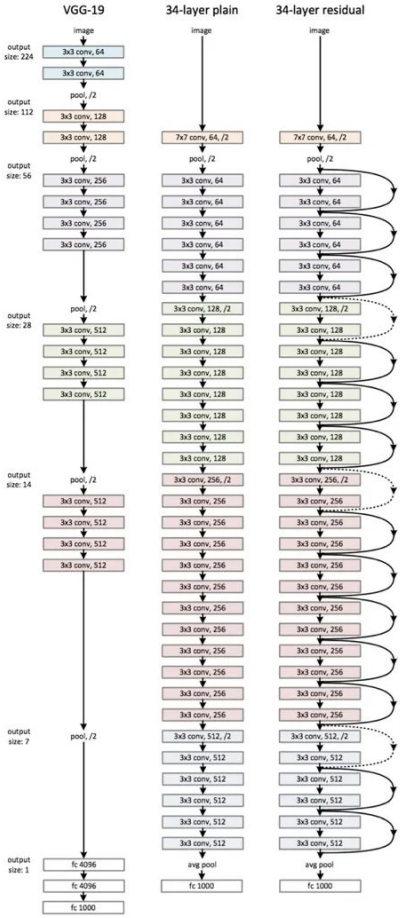


Figure 3. Example network architectures for ImageNet. **Left**: the VGG-19 model [41] (19.6 billion FLOPs) as a reference. **Middle**: a plain network with 34 parameter layers (3.6 billion FLOPs). **Right**: a residual network with 34 parameter layers (3.6 billion FLOPs). The dotted shortcuts increase dimensions. **Table 1** shows more details and other variants.

Pytorch pre-trained model

The following classification models are available, with or without pre-trained weights:

- AlexNet
- ConvNeXt
- DenseNet
- EfficientNet
- EfficientNetV2
- GoogLeNet
- Inception V3
- MaxVit
- MNASNet
- MobileNet V2
- MobileNet V3
- RegNet
- ResNet
- ResNeXt
- ShuffleNet V2
- SqueezeNet
- SwinTransformer
- VGG
- VisionTransformer
- Wide ResNet

Transfer learning:famous CNN architectures

In this topic, you have explored

- Different architecture of CNN models

Topic -5

Transfer learning:with ResNet-18

Transfer learning:with ResNet-18

In this topic, you will explore

- ResNet-18 architecture of CNN models

ResNet architecture

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2				
		$\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$
conv3_x	28×28	$\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times8$
conv4_x	14×14	$\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix}\times6$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times6$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times23$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times36$
conv5_x	7×7	$\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

ures for ImageNet. Building blocks are shown in brackets (see also Fig. 5), with the numbers of block

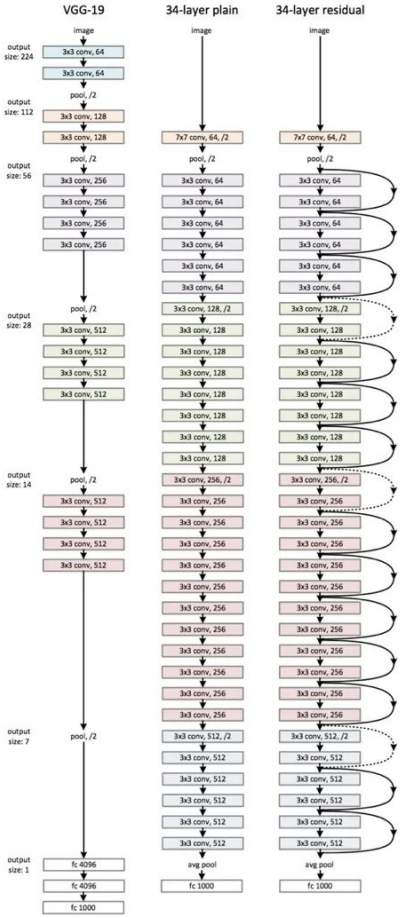
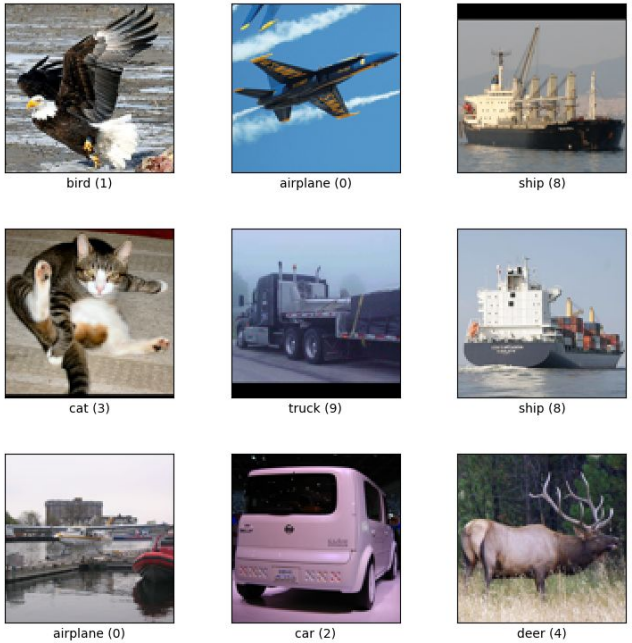


Figure 3. Example network architectures for ImageNet. **Left**: the VGG-19 model [41] (19.6 billion FLOPs) as a reference. **Middle**: a plain network with 34 parameter layers (3.6 billion FLOPs). **Right**: a residual network with 34 parameter layers (3.6 billion FLOPs). The dotted shortcuts increase dimensions. **Table 1** shows more details and other variants.

STL10 datasets



The plan

1. Import the image data
2. Download a pretrained resnet18
3. Freeze the model
4. Replace the final layer
5. Fine-tune only the output layer

A-code

Transfer learning:with ResNet-18

In this topic, you have explored

- Resnet-18 architecture of CNN models

Acknowledgement

Thanks to the following courses and corresponding researchers for making their teaching/research material online

- Mike X Cohen's Master Deep Learning course (most contents taken from this Udemy Course)
- AI Tools such as Chat GPT, Grok, etc
- Introduction to Deep Learning, University of Illinois at Urbana-Champaign
- Introduction to Deep Learning, Carnegie Mellon University
- Convolutional Neural Networks for Visual Recognition, Stanford University
- Natural Language Processing with Deep Learning, Stanford University
- And Many More .....